

El mismo dato, tres veces: XML, JSON y JPA

Clase de teoría con pizarrón, cerrando la Unidad II. El profesor recorre el mismo problema —cómo representar un empleado o un producto fuera de una tabla— con tres formatos distintos: XML "a la antigua", JSON "porque es más simple", y al final el salto real de la clase, JPA, que vuelve a conectar una clase Java con una tabla relacional sin escribir SQL.

Cada bloque lleva el color de su formato: **XML**, **JSON** y **JPA**. Los botones con **?** explican los términos que dio por sabidos. Hay seis ejercicios intercalados.

Lo que salió de esta clase

≈00:00

Práctica nueva: instalar **SonarQube** (Docker recomendado) y llevar el proyecto del conversor de temperaturas a **100 de 100** en cobertura, con pruebas de JUnit y Mockito. Sin fecha fija.

≈00:00

Arrastre: terminar el conversor de temperaturas si aún faltaba (viene de la clase 8). Confía en que ya está.

≈00:00

Examen del primer parcial: anunciado para el **30 de septiembre**, hasta el tema 2.5. La 2.6 (seguridad de servicios) se mueve al siguiente parcial.

cierre

"Nos vemos el jueves. Disfruten su puente." Es la única referencia de fecha explícita en todo el texto.

i

Sobre la fecha y los minutos. No hay marcas de tiempo reales; puse "≈00:00" en el arranque porque esa parte no trae ninguna referencia de minuto. El profesor menciona "hoy estamos a 16, 17, 18" para ubicar el examen "en dos semanas" el 30 de septiembre, lo que sitúa esta clase alrededor del **16 al 18 de septiembre**. Cierra con "nos vemos el jueves, disfruten su puente" — probablemente el puente por el 16 de septiembre. Con la clase 9 puesta el jueves 10, y esta claramente antes del examen del 30 y con un puente de por medio, lo más probable es que esta clase sea el **miércoles 16 de septiembre**. Confírmalo con tu calendario antes de fiarte de esa fecha para otra cosa.

Retoma XML como cierre de lo visto en la clase 7 sobre SOAP y WSDL, y empieza aclarando algo que suena contradictorio.

"XML es considerado como un metalenguaje. En realidad funciona a base de definiciones de metadatos. [...] XML no tiene etiquetas. Existen reglas para crear etiquetas, y las etiquetas ustedes las pueden crear según lo que ustedes consideren."

A diferencia de HTML, que trae un catálogo fijo de etiquetas (`<p>`, `<div>`, `<table>` ...), en XML tú inventas el nombre. Si estás modelando un alumno, tú decides que existan etiquetas como matrícula, nombre o carrera. Lo único que XML impone son las **reglas de forma**, no el vocabulario.

— ¿Qué es un metalenguaje?

Un lenguaje que sirve para describir o construir otros lenguajes. XML no te da un vocabulario de etiquetas: te da las reglas con las que puedes inventar el tuyo (cómo se abre una etiqueta, cómo se cierra, qué es un atributo). Por eso el mismo XML sirve para facturas, para configuración, o para un WSDL: cada quien define su propio conjunto de etiquetas encima de esas mismas reglas.

En su momento se pensó que XML sería el estándar para el intercambio de información entre sistemas heterogéneos, "por su simplicidad en ese momento". A la larga resultó no ser tan simple, porque sus reglas de cumplimiento se volvieron bastante estrictas.

SIGLA	QUÉ ES, SEGÚN ÉL
WSDL	El contrato: cómo llegar a un método, dónde está publicado, qué espera recibir y qué devuelve.
UDDI	"No es la UDDI de allá": sirve para registrar y descubrir qué servicios web existen.

— ¿Qué significa UDDI y por qué aclaró "no es la de allá"?

Universal Description, Discovery and Integration: un directorio donde se publican y buscan servicios web SOAP disponibles, como unas páginas amarillas de servicios. La aclaración es un chiste sobre la coincidencia del nombre con otras siglas o marcas fuera de la informática; no aporta contenido técnico.

BLOQUE 2 · ESTRUCTURA DE UN DOCUMENTO XML

Prólogo, elemento raíz y subelementos

La estructura mínima de cualquier XML, y de dónde puede venir el contenido que se modela.

Todo XML debería tener, como mínimo, una **instrucción de procesamiento** (la declaración de versión y codificación), un **elemento raíz**, y dentro de él, tantos **subelementos** como se necesiten.

```
<?xml version="1.0" encoding="UTF-8"?>
<book>
  <titulo>...</titulo>
  <autor>...</autor>
</book>
```

"¿Quién me dice que la etiqueta `<book>` existe? Pues en realidad se le ocurrió a la persona que definió que iba a modelar un libro."

Con eso ya se cumplen las reglas mínimas. En su momento, XML también se usó para agentes inteligentes y ontologías, "y todo este relajo".

Elementos con solo atributos, y elementos con cuerpo

Partió de un caso concreto: una tabla `Empleado(id, nombre)` de un modelo relacional. Esa misma información en XML se puede repartir entre **atributos y subelementos**:

```
<empleados>
  <empleado id="100">
    <nombre>Juan</nombre>
  </empleado>
</empleados>
```

Aquí `id` es un atributo del elemento `empleado`, y `nombre` es un subelemento suyo. Pero también se puede modelar todo como atributos, sin cuerpo, cerrando la etiqueta en la misma línea:

```
<empleado id="100" nombre="Richard" />
```

"Quiere decir que, cuando se creó la definición del documento XML, se determinó que el elemento empleado solo iba a tener atributos y no iba a tener cuerpo."

— ¿Cuándo conviene un atributo y cuándo un subelemento?

El profesor no dio una regla fija, solo una advertencia: puedes modelar un registro con puros atributos, con puros subelementos, o mezclando los dos, "la que ustedes quieran". La única condición es decidirlo con cabeza, porque quien reciba el XML va a tener que

parsearlo (leerlo con código), y mezclar mucho ambos estilos hace ese parser más complicado. "Se pone divertido más que complejo."

CDATA, comentarios y secuencias de escape

Las secciones **CDATA** guardan contenido tal cual, sin que se interprete como marcado: "no es precisamente un comentario, pero es una formulita". Sirven, por ejemplo, para meter texto con símbolos que de otro modo XML confundiría con etiquetas.

Los comentarios usan la misma sintaxis que HTML, porque HTML la heredó de XML/SGML:

```
<!-- Esto es un comentario -->
```

Para meter caracteres especiales dentro del contenido (como `<` o `>` dentro de una comparación), XML usa **secuencias de escape**, igual que HTML.

— ¿Qué son las secuencias de escape en XML?

Códigos que representan un carácter que, si se escribiera literal, rompería la sintaxis. Por ejemplo `<` representa `<` y `>` representa `>`. El profesor lo ilustró con una condición del estilo "si A es mayor que B y B es mayor que C" (como las que se escriben en C, Java o JavaScript), donde los signos de mayor/menor tendrían que escribirse así para no confundirse con etiquetas.

EJERCICIO 1 · ATRIBUTOS Y SUBELEMENTOS

Leyendo un XML de empleados

Cuatro preguntas sobre la estructura y las reglas de XML que vimos arriba. Contesta las cuatro y dale a **Revisar**.

¿Quién define qué etiquetas existen en un documento XML?

El estándar XML trae un catálogo fijo

Quien lo modela, según lo que quiera representar

Siempre el navegador que lo abre

`<empleado id="100" nombre="Richard" />`: ¿qué tiene y qué no?

Solo atributos, sin cuerpo

Solo un subelemento

Un atributo y un subelemento

¿Qué hace una sección CDATA?

Comenta el contenido para que no se muestre

Guarda contenido tal cual, sin interpretarlo como marcado

Valida el documento contra un XSD

¿De dónde vienen los comentarios de HTML, según el profesor?

Los inventó HTML primero y XML los copió

XML/SGML los tenía primero y HTML los heredó

Son exclusivos de JavaScript

Revisar

Reiniciar

BLOQUE 3 · BIEN FORMADO Y VÁLIDO

Las dos reglas de un XML, y quién las juzga

Retoma la distinción de la clase 7 y la aterriza con un ejemplo de cómo se relaciona con lo que ya sabían de bases de datos.

"Cualquier documento que nosotros queramos escribir con extensión `.xml` se va a mostrar en el navegador, igualito que el HTML. [...] Y no quiere decir que sea un documento válido, simplemente es un documento bien formado."

Un XML tiene que cumplir dos reglas: estar **bien formado** y ser **válido**. Puede estar bien formado sin ser válido. Lo bien formado depende de reglas fijas (elemento raíz único, etiquetas de apertura y cierre que coincidan, nombres de atributo únicos dentro del mismo elemento, etc.). Lo válido depende de un criterio externo.

Ese criterio externo llega por el atributo `xmlns`, que normalmente apunta a una URL. Esa URL puede terminar en `.xsd` o en `.dtd`, y ahí vienen las reglas concretas que dicen si el documento es válido.

— ¿Qué diferencia hay entre un DTD y un XSD?

Los dos describen qué elementos, atributos y estructura debe tener un XML para ser válido contra ese esquema. El **DTD** (Document Type Definition) es el más antiguo y usa su propia sintaxis, no basada en XML. El **XSD** (XML Schema Definition) es más moderno, está escrito en el propio XML, y permite describir tipos de datos con más detalle (enteros, fechas, listas, cardinalidad exacta).

Conectó esto con lo ya visto en bases de datos: las mismas restricciones que se ponen ahí (cardinalidad uno a uno, uno a muchos, listas de subelementos en cierto orden, que aparezca una, varias o muchas veces un subelemento) se pueden expresar en el DTD o el XSD correspondiente.

*"Alguna vez, en el ecosistema de Java, nosotros jugamos con algo que se llamaba **Struts**. [...] Bastaba con ir al DTD, porque venía muy explicadito de qué elementos podíamos tener y para qué servía. Ese fue uno de los proyectos para mi gusto mejor documentados."*

— ¿Qué fue Struts?

Uno de los primeros frameworks Modelo-Vista-Controlador para aplicaciones web en Java, de código abierto. El profesor lo mencionó solo como anécdota, para destacar que su archivo de configuración DTD estaba muy bien documentado.

Cerró la parte de XML con la pregunta que se hace toda esta unidad: ¿se sigue usando? "Sí, para las facturas, de arranque." El resto, no lo sabe.

EJERCICIO 2 · BIEN FORMADO O VÁLIDO

¿Cuál de los dos falla?

Tres casos. Decide si el problema es de buena formación o de validez.

Un XML con dos elementos raíz al mismo nivel.

No está bien formado

Está bien formado pero no es válido

Es válido pero no está bien formado

Un XML con todas sus etiquetas bien cerradas, pero que le falta un subelemento obligatorio según su XSD.

No está bien formado

Está bien formado pero no es válido

Ninguna de las dos cosas se puede saber

¿Qué archivo suele definir esas reglas específicas, apuntado desde `xmlns`?

Un `.css` o `.js`

Un `.xsd` o un `.dtd`

Un `.sql`

Revisar

Reiniciar

JSON · LA ALTERNATIVA SIMPLE

BLOQUE 4 · POR QUÉ JSON

El mismo empleado, mucho más corto

Introduce JSON con una broma sobre el nombre de su ícono habitual, y va directo a la razón de ser: XML funciona, pero es tedioso.

"¿Por qué buscamos otra alternativa? Porque el XML es tedioso, aburrido y propenso a errores. Nada más por eso."

JSON (JavaScript Object Notation) es un formato liviano, una cadena de texto, y resulta natural para quien ya viene programando en JavaScript: modelar un objeto es casi lo mismo que crear una variable.

Escribió primero una versión incorrecta a propósito, para que el grupo la corrigiera:

```
// incorrecto
{
  id: 1,
  nombre: "yo"
}
```

"Miren, mi profe dice algo como esto en clases: equívóquense aquí, y no allá afuera. En el caso de ustedes, equívóquense aquí."

La corrección: los nombres de los atributos van entre comillas dobles, porque un objeto JSON es una lista de pares nombre-valor separados por comas, con dos puntos entre el nombre y el valor.

```
{
  "id": 1,
  "nombre": "yo"
}
```

Comparó el costo de arrancar cada formato: en XML hace falta la declaración de versión y codificación, el elemento raíz, los subelementos... para modelar lo mismo que en JSON basta con unas llaves. "No le resta credibilidad al otro, no le resta poder al otro, simplemente se volvió más simple."



¿Por qué JSON es "más simple para los humanos" y no solo para las máquinas?

No hay etiquetas de apertura y cierre que repetir (como `<nombre>...</nombre>`), no hay que declarar versión ni codificación, y la sintaxis de llaves y corchetes ya es familiar para cualquiera que haya programado en un lenguaje de la familia C o JavaScript. Menos símbolos repetidos, menos oportunidades de error al escribirlo a mano.

BLOQUE 5 · OBJETOS ANIDADOS Y LISTAS

Un JSON dentro de otro, y colecciones

Con la sintaxis básica ya corregida, muestra que JSON aguanta la misma variedad de estructuras que un modelo relacional con asociaciones.

Partió de un caso con dos tablas relacionadas, `evento` y `asistente`, con una asociación uno a muchos entre ellas. Eso se modela metiendo un objeto (o una lista de objetos) dentro de otro objeto:

```
{
  "evento": 1,
  "nombre": "Premiere The Flash",
  "descripcion": "película relacionada con Flashpoint",
  "fecha": "2024-06-14",
  "listaDeAsistentes": [
    { "id": 5, "nombre": "Barry", "paterno": "Allen", "materno": "West",
      "correo": "flash@ligadelajusticia" },
    { "id": 6, "nombre": "Bruno", "paterno": "Díaz", "materno": "",
      "correo": "batman@ligadelajusticia" },
    { "id": 7, "nombre": "Kara", "paterno": "Danvers", "materno": "",
      "correo": "supergirl@ligadelajusticia" }
  ]
}
```

"¿Cómo se llama el atributo? El atributo se llama `listaDeAsistentes`, es igual... es una colección, va entre corchetes, y cada objeto va entre su respectiva llave."

La lectura práctica: si un aplicativo recibe este JSON al crear un evento, puede insertar en la tabla `evento` y, con la lista de asistentes, insertar también en la tabla `asistente`. Un solo objeto JSON puede representar una operación que toca varias tablas relacionadas.

También repasó valores **booleanos** (si algo es administrador, si un documento es descargable), y varios ejemplos rápidos de listas simples: deportes, entidades federativas, niveles de estudio. La conclusión de todos: "¿podemos hacer eso con JSON? Sí."

— ¿Cómo se usa este JSON directamente en JavaScript?

```
let datos = { ... };
console.log(datos.listaDeAsistentes[0].nombre); // "Barry"
```

A diferencia de XML, no hace falta ningún parser aparte: el objeto JSON ya es una estructura de JavaScript en cuanto se asigna a una variable. El profesor lo resumió así: "métenlo en alguna variable y san se acabó. Y operen con él."

Cerró la parte de JSON con una pregunta suelta sobre `var`, `let` y `const`, sin responderla, como algo para que el grupo repase.

EJERCICIO 3 · CORRIGE EL JSON

Encuentra los errores

El mismo ejercicio que hizo en el pizarrón: aquí el JSON tiene errores de sintaxis. Corrígelo y comprueba.

Corrige este JSON

```
{  
  id: 1,
```

Comprobar

Reiniciar

EJERCICIO 4 · ARMA LA LISTA DE ASISTENTES

Objetos dentro de un arreglo

Toca las líneas en el orden en que arma el objeto `evento` con su lista de asistentes. Si te equivocas te doy una pista.

Aquí se va armando el JSON...

```
{  
  "listaDeAsistentes": [  
    "evento": 1,  
    { "id": 5, "nombre": "Barry" },  
    {  
      { "id": 6, "nombre": "Bruno" }  
      "nombre": "Premiere The Flash",
```

Reiniciar

BLOQUE 6 · AVISOS DE GRUPO

SonarQube, el examen y el temario

Cierra la parte teórica y dedica el resto de la clase a tres avisos: una práctica nueva, la fecha del examen, y qué se salta del temario.

La práctica: 100 de 100 en SonarQube

"Necesito que me instalen SonarQube. [...] Si lo instalan a manita, en la máquina en forma local, les va a pedir una base de datos. Trabaja con... de manera natural trabaja con PostgreSQL. Pero le pueden meter con lo que quieran. [...] O mejor aún, para que sea divertido: métenlo en un Docker. Sí, y así repasan contenedores."

El proyecto sobre el que se corre SonarQube es el conversor de temperaturas de la clase 8, "que se supone que ya lo terminaron. Yo confiaré en ustedes." El reto:

"Que en esta cosa, me obtengan, por favor, un 100 de 100. [...] Así como está, si lo echan a andar, van a obtener un cero de 100 en coverage, porque no hay cobertura para pruebas. Hay que escribir pruebas. [...] Pueden agarrar JUnit, Mockito, para hacer pruebas."

Un alumno preguntó si bastaba con probar el "happy path" o también los casos donde la prueba falla. Respuesta: "sorpréndanme". Lo que le interesa es que lo echen a andar y verifiquen que hay cumplimiento. Puso un ejemplo de caso de falla: mandar un carácter en vez de un número al endpoint de conversión.

– ¿Qué es SonarQube y qué mide el "coverage"?

SonarQube es una herramienta de análisis estático de código: revisa tu proyecto y reporta bugs potenciales, vulnerabilidades, código duplicado y, entre otras métricas, la **cobertura de pruebas** (*coverage*): qué porcentaje de las líneas de tu código se ejecuta al menos una vez cuando corres tus pruebas automatizadas. Si no tienes pruebas, la cobertura es 0%, sin importar qué tan bien funcione el código.

– ¿Qué son JUnit y Mockito?

JUnit es el framework estándar de Java para escribir y ejecutar pruebas unitarias: cada prueba llama a un método y compara el resultado contra lo esperado. **Mockito** sirve para crear objetos "de mentiras" (mocks) que simulan una dependencia real, como una base de datos, para poder probar una clase sin depender de que esa pieza externa esté funcionando.

PRÁCTICA ≈00:00 · sin fecha fija

Instalar SonarQube y llegar a 100/100 en el conversor de temperaturas

No dio fecha exacta: "no sé si es para la próxima semana, para dentro de 15 días, para dentro de un mes. Pero háganlo porque lo van a ocupar para el resto del semestre." Se recomienda instalarlo en Docker.

"Necesito que recuerden sus buenas clases de Quality Assurance."

Fecha del examen y el temario que falta

Repasó el avance de la Unidad II: el tema 2.6 (seguridad de servicios) se pospone hasta que haya servicios con acceso real a datos, roles y permisos, porque "esa se aguanta". El 2.2 se cierra con esta clase. El 2.3 (principios arquitectónicos de un API REST) ya se hizo en la práctica, aunque falta la teoría. Decidió saltarse la teoría de 2.3, y el 2.4 y 2.5 completos, "porque si no, no nos va a alcanzar el tiempo", con intención de volver a ellos después.

"El examen solo va a venir hasta la 2.5, por lo tanto lo tengo que ver antes de que termine este parcial. La 2.6 la voy a mover para el otro periodo. [...] Hoy estamos a 16, 17, 18... Ah, okay, más bien en dos semanas: 28, 29 y 30. Entonces vamos a dejar el examen para el 30."

FECHA CONFIRMADA 30 de septiembre

Examen del primer parcial

Cubre hasta el tema 2.5 de la Unidad II. El 2.6 (seguridad de servicios) queda fuera y se ve en el siguiente parcial.

"Se traen muchas ganas, bolígrafo, corrector, y demás cosas."

Antes de pasar al bloque de JPA, dejó escrito en el pizarrón el arranque de la práctica que viene: crear una base de datos.

```
CREATE DATABASE productos_CM2;
```

EJERCICIO 5 · AVISOS DE LA CLASE

SonarQube, el examen y el temario

Cuatro preguntas sobre lo anunciado en este bloque.

¿Qué proyecto se somete a SonarQube?

El conversor de temperaturas de la clase 8

El controlador de productos de la clase 9

Un proyecto nuevo, todavía sin nombre

Sin escribir ninguna prueba, ¿qué cobertura (*coverage*) va a marcar SonarQube?

100%, porque el código funciona

0%, porque no hay pruebas que ejecuten el código

50%, un punto de partida típico

¿Qué tema del temario se pospone hasta que haya servicios con acceso real a datos, roles y permisos?

2.3, principios arquitectónicos de REST

2.6, seguridad de servicios

2.1, introducción a APIs

¿Hasta qué tema entra el examen del primer parcial, fijado para el 30 de septiembre?

Hasta el 2.5

Hasta el 2.6

Hasta el final de la Unidad II

Revisar

Reiniciar

JPA · EL OBJETO VUELVE A LA TABLA

BLOQUE 7 · EL MODELO DE LA TABLA PRODUCTO

De la tabla a la clase Java, sin anotaciones todavía

Retoma la tabla `CREATE DATABASE productos_CM2` del bloque anterior y empieza a modelar la tabla `Producto`, primero en SQL y luego como clase Java corriente.

COLUMNA	TIPO	RESTRICCIONES
id_producto	INT	NOT NULL, AUTO_INCREMENT, PRIMARY KEY
nombre	VARCHAR(50)	NOT NULL
descripcion	VARCHAR(200)	NOT NULL
precio	DOUBLE	NOT NULL. Buscaba el tipo específico para moneda y no lo recordó; se quedó con <code>double</code>
existencia	INT	NOT NULL

— ¿Cuál es "el tipo específico para moneda" que no recordó?

En SQL Server existe el tipo `MONEY`, y varios motores tienen `DECIMAL(p, s)` o `NUMERIC`, que guardan un número exacto con parte entera y fraccionaria sin el redondeo binario de `double`. Para dinero real conviene `DECIMAL` antes que `double`, precisamente porque los flotantes pueden acumular error de redondeo. El profesor lo dejó como `double` "para no complicar" el ejemplo.

Con la tabla ya definida, escribe la clase Java "muy naturalita", sin patrones de diseño todavía:

```
public class Producto implements Serializable {
    private int id_producto;
    private String nombre;
    private String descripcion;
    private double precio;
    private int existencia;

    // @Data @NoArgsConstructor @AllArgsConstructor @Builder van aquí
}
```

"Imagínense que va un `@Data`, un `@NoArgsConstructor`, un `@AllArgsConstructor`, y a lo mejor un `@Builder`. [...] Nuestra clase tiene un constructor con argumentos cero, un constructor con todos los parámetros, un montón de getters, un montón de setters, un `toString`, un `equals`, un `hashCode`, un `canEqual`, y el `clone` y lo que ustedes quieran."

- i **Nota mía.** Es la misma jugada de la clase 9 con `Producto`: primero se escribe todo a mano (o se da por sabido) y luego se resume con Lombok. Aquí ni siquiera lo vuelve a escribir en el pizarrón; da por hecho que ya saben ponerlo. Ojo: `@Data` no genera `canEqual` ni `clone` por sí solo — `canEqual` sí lo agrega automáticamente junto con `equals` / `hashCode`, pero `clone` no es parte de Lombok y habría que escribirlo aparte si de verdad se necesita.

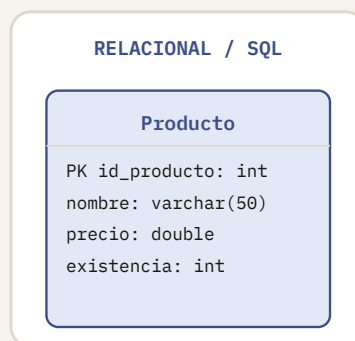
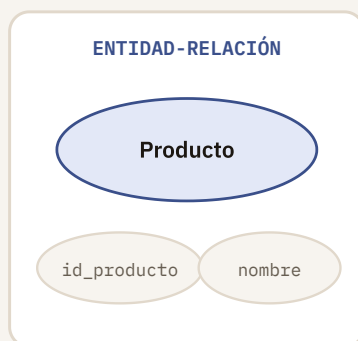
BLOQUE 8 · EL PROBLEMA QUE RESUELVE JPA

Modelo Entidad-Relación, Relacional, y por qué un objeto no se inserta solo

Antes de llegar a las anotaciones, plantea el problema de fondo: ¿cómo casa una clase orientada a objetos con una tabla?

"¿Cómo se llama este modelo? [...] Entidad-Relación. Nada más es una entidad, todavía no tiene relaciones."

Repasó la cadena que ya conocían de Bases de Datos: **Entidad-Relación** primero, **Modelo Relacional** después (con la notación de llave primaria y tipos de columna), y de ahí el **SQL** que crea la tabla real. En Entidad-Relación, la tabla es una **Entidad** y sus columnas son **Atributos**.



Tres formas de ver el mismo producto. JPA es el puente entre la de la derecha y la del centro: sin él, no hay forma directa de pasar de una instancia de objeto a una fila de tabla.

"¿Puedo meter una instancia de un objeto en esta cosa? No, o no directamente. [...] Si yo creo la instancia de Producto y le digo

`Producto p = new Producto(...)`, lo lleno, y voy a decir 'p, insértate', no se puede."

Recordó cómo se hacía antes, en el CRUD manual (como el de la clase 9): ibas parámetro por parámetro, llamando a cada *getter* para sacar el valor

primitivo correspondiente y armando el `INSERT` a mano. Eso es exactamente lo que JPA promete evitar.

— ¿Qué es ORM y por qué se llama "mapeo"?

ORM (Object-Relational Mapping): la técnica de hacer corresponder ("mapear") clases orientadas a objetos con tablas relacionales, y atributos de esas clases con columnas de esas tablas. La palabra "mapeo" viene de esa correspondencia: dado un objeto, el ORM sabe a qué fila de qué tabla corresponde, sin que el programador escriba el

`INSERT` / `SELECT` a mano.

"Da lo mismo con qué estemos programando, mientras sea orientado a objetos necesito casarlo con mi tablita."

EJERCICIO 6 · EL PROBLEMA DE FONDO

Por qué hace falta un ORM

Tres preguntas sobre la idea central de este bloque, antes de ver las anotaciones concretas.

¿Se puede insertar directamente una instancia de `Producto` en la tabla, sin ningún mecanismo intermedio?

Sí, Java lo hace automáticamente

No: no hay forma directa de pasar de un objeto a una fila

Sí, pero solo si implementa `Serializable`

¿Cómo se resolvía este problema antes de JPA, con el CRUD manual (como el de la clase 9)?

Iban getter por getter, armando el `INSERT` a mano con cada valor primitivo

No se podía resolver de ninguna forma

Java lo convertía solo con `toString()`

¿Qué nombre recibe la técnica de hacer corresponder clases orientadas a objetos con tablas relacionales?

DAO

ORM

MVC

Revisar

Reiniciar

BLOQUE 9 · LAS ANOTACIONES DE JPA

`@Entity`, `@Table`, `@Id`, `@GeneratedValue` y `@Column`

Con el problema ya planteado, va anotación por anotación, construyendo la clase `Producto` real, la que sí conecta con la tabla.

Primero, decirle a Java que esta clase representa una entidad:

```
@Entity
```

"Mira, Producto, estoy modelando esta entidad, por lo tanto es un `@Entity`."

Después, decir a qué tabla concreta corresponde, por si el nombre de la clase no coincide con el nombre de la tabla:

```
@Table(name = "producto")
```

Adoptó una regla propia para simplificar el resto de la explicación: "tal cual se llama mi entidad, tal cual se llama mi tabla, tal cual se llama mi clase" — aunque en la práctica `@Table` existe justo para el caso en que eso no se cumpla (como una clase llamada `Productos`, en plural, apuntando a una tabla `producto`, en singular).

Sobre el campo llave, tres anotaciones trabajando juntas:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id_producto", nullable = false)
private long id_producto;
```

- `@Id` : marca el atributo como llave primaria.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` : le dice al manejador de la base que él genera el autonumérico. "Yo no voy a generar el ID porque tengo flojera, va a ser el manejador."
- `@Column(name = "id_producto", nullable = false)` : hace explícito el nombre real de la columna y que es obligatoria (el `NOT NULL` de la tabla).

*"Aquí sí, así como en SQL no es estándar para todo, sobre todo en generación de autonuméricos: por ejemplo Postgres utiliza un tipo de dato serial, en realidad es una secuencia. Ahí tendría que cambiar, porque tendría que ser un **SEQUENCE**, y tendríamos que poner un `@SequenceGenerator`."*

— ¿Por qué cambia la estrategia según el motor de base de datos?

`IDENTITY` funciona bien con motores como MySQL, que generan el autonumérico directamente en la columna. `SEQUENCE` es el mecanismo que usan motores como PostgreSQL u Oracle, donde el autonumérico en realidad es un objeto de base de datos aparte (una secuencia) del que se pide el siguiente valor. JPA soporta ambas estrategias, pero hay que elegir la que corresponda al motor real que estés usando.

Cerró remarcando cuándo es indispensable `@Column(name = ...)`: solo cuando el nombre del atributo de Java no coincide exactamente con el de la columna. Si coinciden, JPA hace el mapeo automáticamente y esa anotación es opcional.

"Ahora sí le estamos diciendo: 'ah, mira, hay una tabla en mi base de datos que se llama producto, que tiene un atributo que se llama id_producto, que es obligatorio, que es autoincrementable y que es mi llave primaria'. Se acabó."

i **La clase se corta aquí.** El profesor terminó la clase justo después de explicar el campo `id_producto` completo, con "nos vemos el jueves, disfruten su puente". No llegó a poner `@Column` en `nombre`, `descripcion`, `precio` ni `existencia`; queda como ejercicio natural para la siguiente sesión, aplicando la misma regla que para el id.

EJERCICIO 7 · ARMA EL ATRIBUTO `ID_PRODUCTO`

Las tres anotaciones, en orden

Toca las líneas en el orden en que las escribió, de arriba abajo.

Aquí se va armando el atributo...

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

`@Column(name = "id_producto", nullable = false)`

`private long id_producto;`

`@Id`

Reiniciar

EJERCICIO 8 · ESCRIBE LA ANOTACIÓN QUE FALTA

@Column para los otros cuatro atributos

Escribe la anotación completa para cada atributo, siguiendo el mismo patrón que usó para `id_producto`. Solo cambia lo que corresponde: el nombre de columna y si es obligatorio.

Atributo `nombre` (VARCHAR(50), NOT NULL)

Comprobar

Atributo `descripcion` (VARCHAR(200), NOT NULL)

Comprobar

Atributo `precio` (DOUBLE, NOT NULL)

Comprobar

Atributo `existencia` (INT, NOT NULL)

Comprobar

Reiniciar

CÓDIGO DE LA CLASE

La entidad completa

La clase `Producto` tal como quedaría si se completa el patrón que dejó a medias, con Lombok y las cuatro anotaciones de JPA que sí explicó.

```

package com.ipn.mx.modelo;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "producto")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_producto", nullable = false)
    private long id_producto;

    @Column(name = "nombre", nullable = false)
    private String nombre;

    @Column(name = "descripcion", nullable = false)
    private String descripcion;

    @Column(name = "precio", nullable = false)
    private double precio;

    @Column(name = "existencia", nullable = false)
    private int existencia;
}

```

– ¿En qué se diferencia esta versión de lo que quedó en el pizarrón?

Son cabos sueltos del ritmo de la clase, no correcciones a lo que dijo:

- **@Column** en los otros cuatro atributos. No llegó a escribirlos; los completé siguiendo la misma regla que usó para `id_producto` (nombre igual al atributo, todos obligatorios porque la tabla los declaró `NOT NULL`).
- **implements Serializable**. Lo puso en la versión "natural" sin anotaciones, antes de introducir JPA. Con `@Entity`, `Serializable` ya no es obligatorio (aunque sigue siendo buena práctica si la entidad viaja entre capas), y no lo repitió al hablar de las

anotaciones. Lo dejé fuera de esta versión final para no inventar una decisión que no tomó en voz alta.

- **Imports y paquete.** No aparecen en la transcripción; agregué los de `jakarta.persistence` (la especificación moderna de JPA) para que compile.
- **Tipo del id.** Dijo "pudo haber sido un `Integer` o un `int`", y lo voy a dejar en `long`", pero el pizarrón de la versión "natural" (antes de JPA) lo tenía como `int id_producto`. Aquí lo dejé como `long`, que es lo que explícitamente decidió al llegar a esa línea.

CIERRE

Checklist de la clase 10

Se guarda en tu navegador. Nada se sube a ningún lado ni se comparte con nadie más.

0 / 4

☐ Práctica · instalar SonarQube y llegar a 100/100 en el conversor

Instálalo de preferencia en Docker. Necesitas pruebas con JUnit y Mockito para subir la cobertura del 0%; sin fecha fija, pero se usa el resto del semestre.

☐ Examen del primer parcial · 30 de septiembre

Cubre hasta el tema 2.5 de la Unidad II. La 2.6 (seguridad de servicios) queda para el siguiente parcial.

☐ Completar la entidad Producto

Faltan las anotaciones `@Column` en nombre, descripcion, precio y existencia; la clase se cortó justo después de terminar `id_producto`.

☐ Arrastre · conversor de temperaturas (clase 8)

El profesor asume que ya está terminado. Si te falta alguna conversión, es la base sobre la que corre la práctica de SonarQube.